Apt Store - Order Management System

TABLE OF CONTENT:

Overview

Apt Store is a full-stack order management system with real-time tracking. It supports user reviewer, and admin, each with a dedicated interface.

Users can place orders and track their status live via **WebSocket**. As soon as a reviewer updates an order status, the change is pushed instantly to the user's dashboard without a page refresh. The WebSocket connection is managed automatically, it opens when the user has active orders and closes when none remain.

Reviewers get a dashboard showing all orders with an inline status editor, click an order, pick a new status, save.

Admins are redirected directly to the Django admin panel.

The backend is built with **Django REST Framework** for the REST API and Django Channels for WebSocket support, running on Daphne as the ASGI server. The frontend is a **React** SPA using session-based authentication with CSRF protection.

Tech Stack

1. Backend

- Python / Django Rest Framework
- Django Channels (WebSockets)
- Daphne (ASGI server)
- SQLite

2. Frontend

- React 18
- React Router v6
- Axios
- Vite

Database Schema

The project uses two main models: User (custom) and Order.

1. **User:** Replaces Django's default user model via a custom AbstractBaseUser.

Field	Type	Description
id	Integer (PK)	Auto-incremented primary key
name	CharField	Full name of the user
email	EmailField	Unique, used as login identifier instead of username
phone	CharField	Contact number
password	CharField	Hashed password
role	CharField	One of user, reviewer, admin
is_active	BooleanField	Whether account is enabled (default: True)
is_staff	BooleanField	Whether user can access Django admin
is_superuser	BooleanField	Full admin privileges

Notes:

- email is the *USERNAME_FIELD*, used for authentication instead of username
- *REQUIRED_FIELDS* are defined via a custom manager in manager.py
- Password is never stored in plain text, Django handles hashing automatically
- The system refers to these roles in registration and permission logic: **(user, reviewer, admin)**

2. Order: Represents a single order placed by a user.

Field	Type	Description
id	Integer (PK)	Auto-incremented primary key
product_name	CharField	Name of the product ordered
status	CharField	Current status of the order
user	ForeignKey → User	The user who placed the order

Status choices

Value	Meaning
pending	Order placed, not yet shipped
shipped	Order dispatched
delivered	Order received by user
cancelled	Order cancelled

- Default status on creation is pending.
- Notes:
  - user is set automatically on the backend from request.user, frontend never sends it
  - An order is considered active if its status is pending or shipped, this drives the WebSocket connection logic
  - Deleting an order is only meaningful when status is pending

API Documentation

Base paths:

- Accounts base path: **/accounts/auth/**
- Orders base path: **/orders/**
- WebSocket path: **ws://localhost:8000/ws/orders/**

Authentication model:

This backend uses Django session based authentication. A successful login calls *login(request, user)*, which creates a server-side session and returns a success message together with a serialized user object. The browser should preserve and send the session cookie on later authenticated HTTP requests and on the WebSocket handshake.

1. Accounts API

1.1 Register

Method: POST

URL: /accounts/auth/register/

Permission:AllowAny

Request body

```
{
    "name": "Alice",
    "email": "alice@example.com",
    "phone": "9876543210",
    "password": "strong-password",
    "role": "user"
}
```

Success response

Status: 201 Created

```
{
    "message": "Account created successfully."
}
```

Validation error response

Status: 400 Bad Request

Shape depends on serializer validation errors. Example:

```
{
    "email": ["This field must be unique."],
    "password": ["This field is required."]
}
```

1.2 Login

Method: POST

URL: /accounts/auth/login/

Permission:AllowAny

Request body

```
{
    "email": "alice@example.com",
    "password": “strong-password"
}
```

Success response

Status: 200 OK


```
{
    "message": "Logged in successfully.",
    "user": {
        "id": 7,
        "name": "Alice",
        "email": "alice@example.com",
        "phone": "9876543210",
        "role": "reviewer"
        }
}
```


Error responses

Status: 400 Bad Request, serializer validation failed


```
{
    "email": ["This field is required."]
}
```


Status: 401 Unauthorized, invalid credentials


```
{
    "error": "Invalid email or password."
}
```


Status: 403 Forbidden, user exists but account is disabled


```
{
    "error": "Account is disabled.”
}
```


1.3 Logout

Method: POST

URL: /accounts/auth/logout/

Permission: IsAuthenticated

Request body: No body required.

Success response

Status: 200 OK


```
{
    "message": "Logged out successfully."
}
```


2. Orders API

Permission behavior: The provided order views use custom permissions are

- IsUserOrReviewer on list, retrieve, create, and delete routes through OrderViewset
- IsReviewer on the update route through OrderUpdateViewset

2.1 List orders

Method: GET

URL: /orders/list/all/

Permission: IsUserOrReviewer

Behavior:

- If the logged-in user is a reviewer or admin, the backend returns all orders.
- Otherwise, it returns only orders belonging to the current user.

Success response

Status: 200 OK


```
[
    {
    "id": 12,
    "product_name": "Keyboard",
    "status": "pending",
    "user": 7
    }
]
```

2.2 Retrieve order

Method: GET

URL: /orders/retrieve/<orderId>/

Permission: IsUserOrReviewer

Path params

- orderId: order primary key

Success response

Status: 200 OK

```
{
    "id": 12,
    "product_name": "Keyboard",
    "status": "pending",
    "user": 7
}
```

Not found response

Status: 404 Not Found

```
{
    "error": "Order not found"
}
```

2.3 Create order

Method: POST

URL: /orders/create/

Permission: IsUserOrReviewer

Request body

```
{
    "product_name": "Keyboard"
}
```

Success response

Status: 201 Created

```
{
    "message": "Order created succesfully”
}
```

Validation error response

Status: 400 Bad Request

Example shape:

```
{
    "product_name": ["This field is required."]
}
```

2.4 Delete order

Method: DELETE

URL: /orders/delete/<orderId>/

Permission: IsUserOrReviewer

Path params

- orderId: order primary key

Success response

Status: 200 OK

```
{
    "message": "Order deleted succesfully."
}
```

Not found response

Status: 404 Not Found

```
{
    "error": "Order not found."
}
```

2.5 Update order status

Method: PATCH

URL: /orders/update/<orderId>/

Permission: IsReviewer

Path params

- orderId: order primary key

Request body

```
{
    "status": "shipped"
}
```

Allowed status values

- pending
- shipped
- delivered
- cancelled

Success response

Status: 200 OK

```
{
    "message": "Order status updated successfully."
}
```

Validation error response

Status: 400 Bad Request

Example shape:

```
{
    "status": ["\"processing\" is not a valid choice."]
}
```

Not found response

Status: 404 Not Found

```
{
    "error": "Order does not found."
}
```

WebSocket API

URL: ws://localhost:8000/ws/orders/

Authentication behavior

The consumer reads **self.scope["user"]** and rejects unauthenticated users. In practice, the browser must include the authenticated session cookie during the WebSocket connection.

Connect outcomes:

Case 1: unauthenticated user

The server closes the socket immediately.

- Close code: 4001
- No successful session is established.

Case 2: authenticated user with no active orders

The server accepts the connection, sends a closing message, and then closes the socket.

Message:

```
{
    "type": "connection.closing",
    "reason": "No active orders"
}
```

Then it closes with code 1000.

Case 3: authenticated user with active orders

The server accepts the connection, adds the socket to user_<user.id>_orders, and immediately sends all current active orders.

Initial message:

```
{
    "type": “active_orders",
    "orders": [
        {
            "id": 12,
            "product_name": "Keyboard",
            "status": "pending"
        }
    ]
}
```

Active order definition: An order is considered active if its status is in ‘pending’ or ‘shipped ’ state.

Status update message

When a reviewer updates an order status successfully, the order owner receives this WebSocket event:

```
{
    "type": "order.status",
    "order_id": 12,
    "status": "shipped",
    "product_name": "Keyboard"
}
```

Auto-close behavior

An order.status message, the consumer checks whether the user still has any active orders.
If no active orders remain, the server sends:

```
{
    "type": "connection.closing",
    "reason": "No active orders remaining."
}
```

Then it closes the connection with code 1000.